Computer Science & IT



Database Management System

Transactions and Concurrency control

Lecture No. 08



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Concurrency control protocols
- **Topic** Simple use of shared and exclusive locks
- **Topic** Basic Two phase lock (2PL)

Slide

Topics to be Covered



- Topic: Basic 2PL and lock upgrading/downgrading
- Topic: Problems possible with Basic 2PL
- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL

Slide



Topic : Lock Point

The Point at which the growing phase ends, i.e., the point at which transaction takes the final lock it needs to carry on its work.

eg:

S

T_1	T_2
X(A)	
R(A)	
W(A)	
X(B)	
U(A)	
	S(A)
	R(A)
R(B)	
W(B)	
U(B)	
	S(B)
	R(B)
	U(A)
	U(B)

lock point in T_1

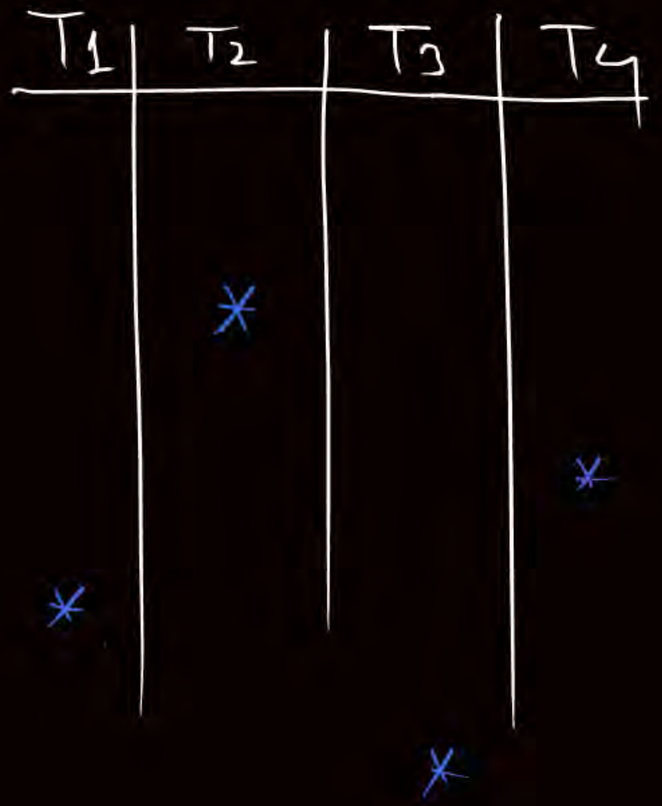
lock point in T_2

It is a C.S.S. which is allowed by 2PL

W.r.t. given schedule
Order of lock point is $T_1 \longrightarrow T_2$

∴ Conflict equivalent Serial schedule is $T_1 \longrightarrow T_2$

eg: Consider the following schedule is allowed to execute using 2PL, and "*" represent the lock points in the respective transactions



Conflict equivalent serial schedule will be

$T_2 \longrightarrow T_4 \longrightarrow T_1 \longrightarrow T_3$



Topic : Basic 2PL with lock upgrading

If lock conversion (upgrading/downgrading) is allowed, then we can upgrade shaded lock into exclusive locks in the Growing Phase, and we can downgrade from the exclusive lock to shared lock in the shrinking phase.

If not already in the shrinking phase, then the moment we perform downgrade op^{n} shrinking phase of that transaction will begin.

Topic : Basic 2PL with lock upgrading

2PL Without Lock upgrading

eg.

T_1	T_2
	X(B)
	R(B)
S(A)	
R(A)	
~~S(B)~~ denied	
R(B)	
	W(B)

Schedule is a C.S.S. but it is not allowed to execute using 2PL Without lock upgrading

2PL With lock upgrading

T_1	T_2
	S(B) ← because Currently we want to perform Read op^n
	R(B)
S(A)	
R(A)	
S(B)	
R(B)	
U(A)	
U(B)	
	X(B)
	W(B)

Shared lock can be upgraded into exclusive lock because T_2 has not performed any Unlock op^n

This C.S.S. can be allowed to execute by 2PL if lock upgrading is allowed

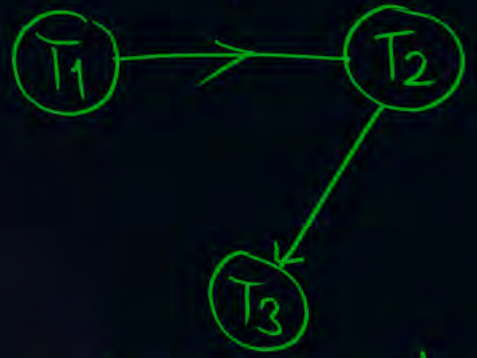
Topic : Basic 2PL with lock upgrading

Without lock upgrading

eg:

T_1	T_2	T_3
	X(A)	
	R(A)	
X(B)		
R(B)		
	W(A)	
	~~X(B)~~ denied	
	U(A)	X(A)
		R(A)
W(B)		
		W(A)
	R(B)	
	W(B)	

Precedence graph
w.r.t. Conflict serializability



Acyclic precedence graph
∴ Conflict serializable schedule

It is a Conflict Serializable schedule and not allowed to execute using 2PL without lock upgrading

Topic : Basic 2PL with lock upgrading



With lock upgrading

eg:

T_1	T_2	T_3
	S(A)	
	R(A)	
✓ S(B)		
R(B)		
	X(A) ← it can be upgraded	
	W(A)	
	X(B) denied {If we req. for S(B) at this point	
	U(A)/S(A)	
	S(A)	
	R(A)	
W(B)		
	W(A)	
R(B)		
W(B)		

then it can not be upgraded to Exclusive lock because of

This C.S.S. is neither allowed using without lock upgrading nor allowed with lock upgrading

Note:- If lock upgrading is allowed with 2PL, then it may execute some more conflict serializable schedule, but still there will be some conflict serializable schedule which are not allowed 2PL even with lock upgrading

Note: If schedule is not a C.S.S, then it is never allowed using 2PL, no matter lock upgrading is allowed or not.



Topic : Problems possible with Basic 2PL

A schedule that is allowed to execute using basic 2PL protocol may suffer from,

(1) Irrecoverability [Irrecoverability is possible that means uncommitted read op^n are allowed, and hence Cacading rollback is also possible]

(2) Deadlock is possible

(3) Starvation is possible



Topic : Irrecoverability with Basic 2PL

eg:

T_1	T_2
X(A)	
R(A)	
W(A)	
X(B)	
X(C)	
U(A)	
	S(A)
	R(A)
R(B)	
W(B)	
U(B)	
	S(B)
	R(B)
	U(A)
	U(B)
	Commit
R(C)	
U(C)	
Commit	

Uncommitted Read

Committed before T_1

∴ Irrecoverable

✓ Schedule is allowed to execute using 2PL;

but it is an irrecoverable schedule

{ And uncommitted read oph exists

∴ Cascading rollback problem exists }

Solution of this problem will be "Strict-2PL"

Topic : Deadlock with Basic 2PL



eg:

T_1	T_2
S(A)	
R(A)	
	X(B)
	W(B)
denied S(B)	
R(B)	
T.O.	
X(A) denied	
W(A)	
T.O.	

T_1 depends on T_2 to unlock B

Dependency graph

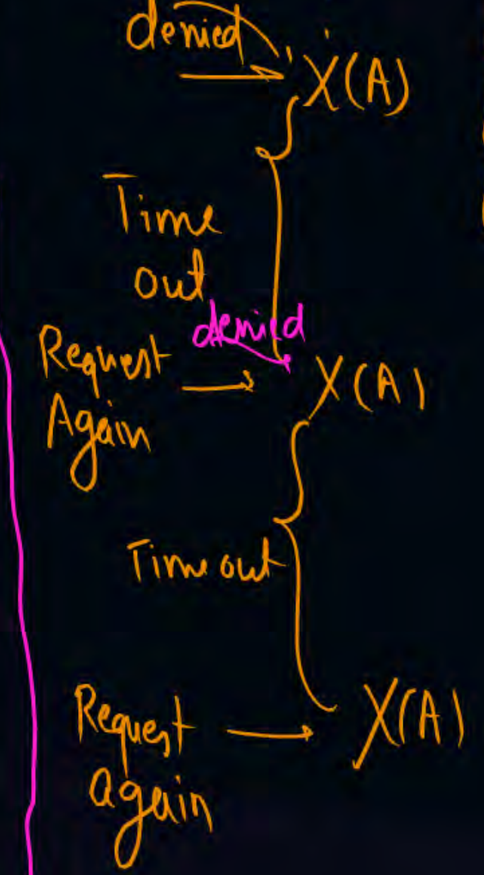


T_2 depends on T_1 to unlock A

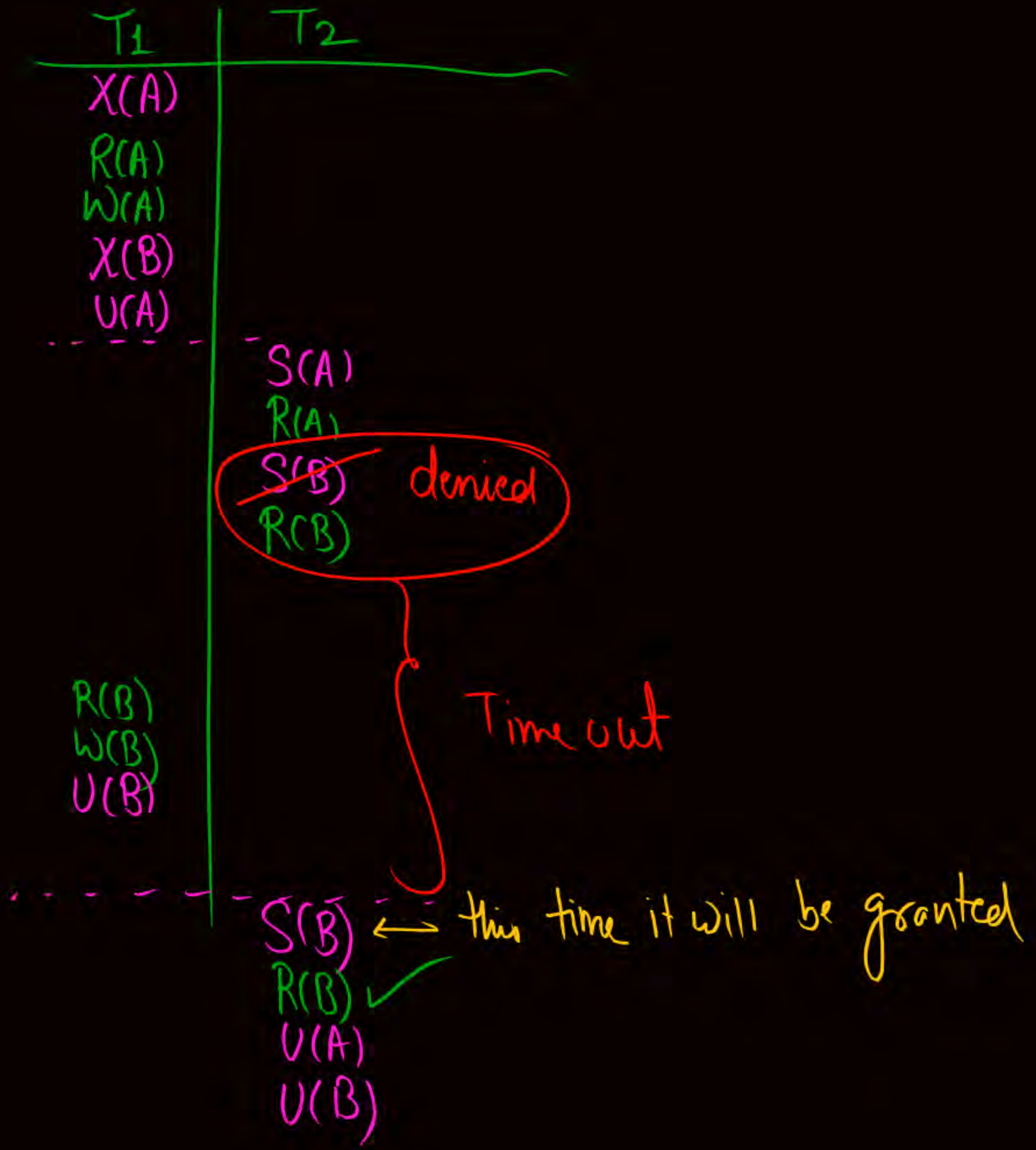
Cyclic dependency graph
∴ Deadlock

Solution for this problem is Conservative 2PL

Transaction

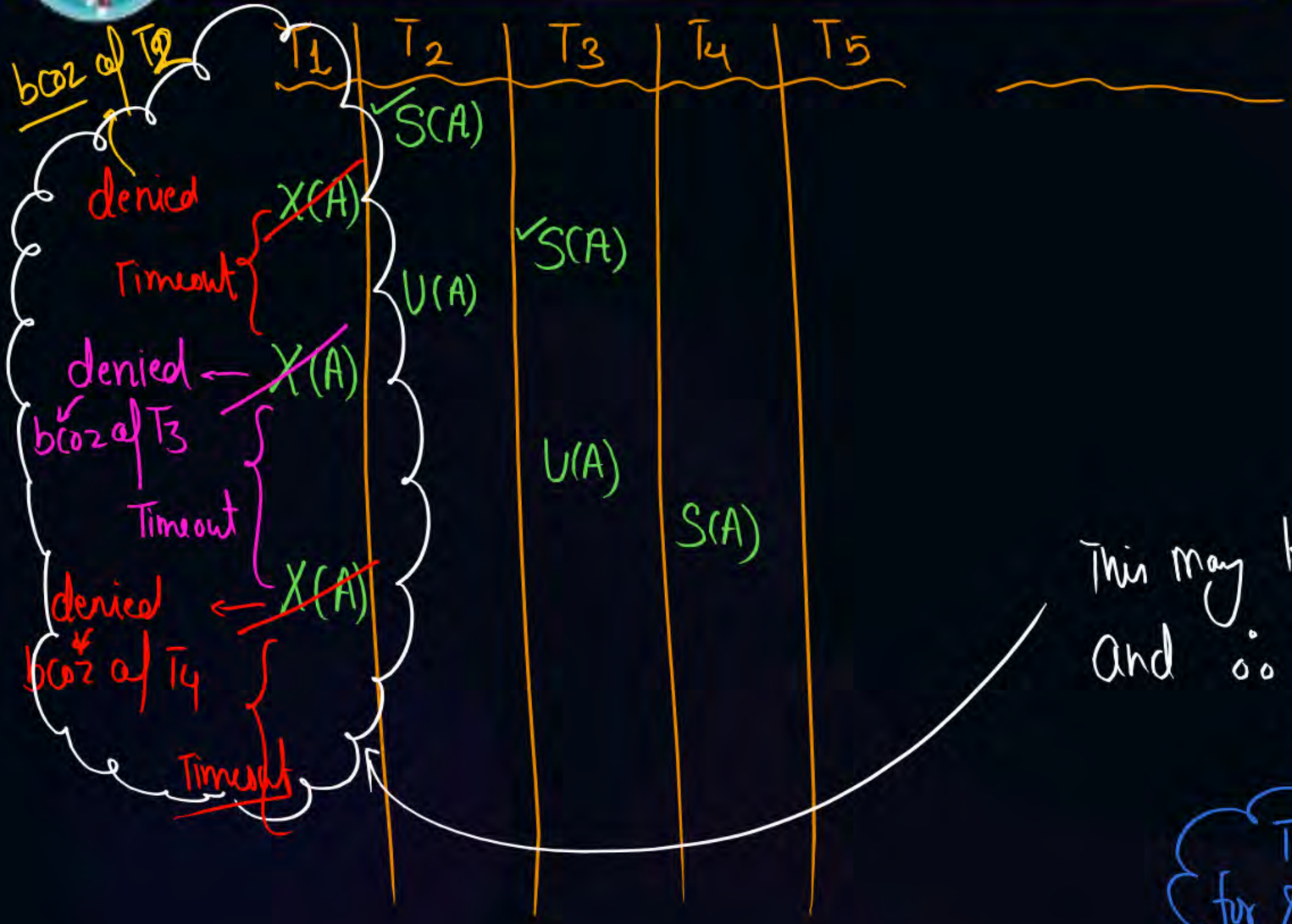


If transaction request for a lock and if the lock is denied then transaction will go into a time out period and after that time out period it will again request for that lock


T1
T2
X(A)
R(A)
W(A)
X(B)
U(A)
S(A)
R(A)
S(B)
denied
R(B)
Time out
R(B)
W(B)
U(B)
S(B)
this time it will be granted
R(B)
U(A)
U(B)



Topic : Starvation with Basic 2PL



This may keep happening and ∴ Transaction T_1 will starve for Exclusive lock on data item A.

There is no solution for Starvation using locking Protocols.



Topic : Classification of Schedules based on 2PL

Basic 2PL

Strict 2PL

Conservative 2PL

Rigorous 2PL

{ Already done }



2 mins Summary

- Topic: Basic 2PL and lock upgrading/downgrading
- Topic: Problems possible with Basic 2PL
- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL

Slide

THANK - YOU

Slide